Stable and Efficient Benchmarking via Information-Theoretic Feature Selection

Lab meeting April 2026

Problem setting

So many evals, so little time

Nice if we can run subsampled versions of benchmarks to get accurate full-benchmark performance with less time/compute/money

-> Especially if the benchmark is slow, e.g. CoT, code execution, multi-turn, multiple answer generation (e.g. for pass@k)

-> And if you want to run evals for lots of ablations (varying temp, min_p, prompt format etc.)

Problem setting

Our approach Let $\mathcal{D}$ be a benchmark dataset containing $|\mathcal{D}| = N$ questions, and let $s(m, q_i)$ be the score achieved by model m on question q_i, $i \in [N]$. Given the per-question results from M *source-models*, $\{s(m, q_i)\}_{m \in [M], i \in [N]}$, we can define the task efficient benchmarking in two stages:

Stage 1: Select a *coreset* of questions $\mathcal{C} \subset \mathcal{D}$ of size $|\mathcal{C}| = n < |\mathcal{D}| = N$.

Stage 2: Learn a function $f_{\mathcal{C}} : \mathbb{R}^n \to \mathbb{R}$ that predicts the full-benchmark score of a test model using only the coreset question-scores: $f_{\mathcal{C}}([s(m^*, q_i)]_{q_i \in \mathcal{C}}) \approx \bar{s}(m^*, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} s(m^*, q_i)$.

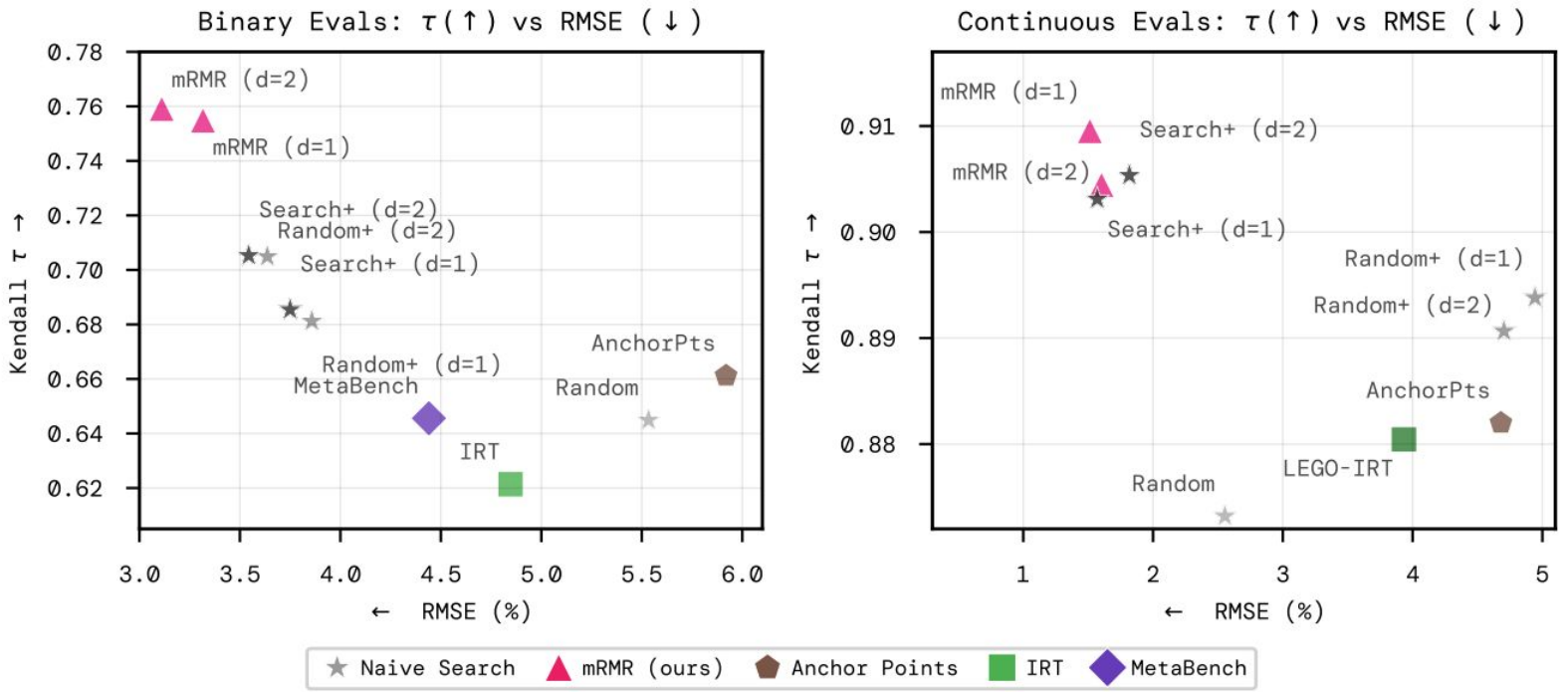
Stage 1 = feature selection

-> Use mRMR (minimum redundancy maximum relevance)

Stage 2 = multiple regression (only using coreset questions)

-> Use (kernel) ridge regression

Our approach



mRMR

Features X_1, …, X_n can be used to predict Y

We want a subset (*coreset*) of n features, C, with minimum redundancy and maximum relevance

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y), \qquad \text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j).$$

Where I is the mutual information

$$I(X,Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X) + H(Y) - H(X,Y).$$

$$I(X,Y) = D_{KL}(P_{X,Y} \mid\mid P_X \otimes P_Y) = \int_{\Omega_X} \int_{\Omega_Y} p(x,y) \log \left(\frac{p(x,y)}{p(x)p(y)} \right) dydx \geq 0.$$

mRMR Greedy Approach

$$\text{Relevance}(\mathcal{C}) = \frac{1}{n} \sum_{X_i \in \mathcal{C}} I(X_i, Y), \qquad \text{Redundancy}(\mathcal{C}) = \frac{1}{\binom{n}{2}} \sum_{X_i \in \mathcal{C}} \sum_{X_j \in \mathcal{C} \setminus \{X_i\}} I(X_i, X_j).$$

Finding the optimal coreset of size n (out of N) is NP-Complete -> greedy approach with feature importance function, g.

Algorithm 1: Greedy mRMR Feature Selection

Input : Dataset of features $\mathcal{D} = \{X_1, \ldots, X_N\}$, Target variable Y, Desired subset size n
Output : Feature set/Coreset $\mathcal{C}$

// Step 1: Selection of the first feature (Maximum Relevance)
$X^* \leftarrow \arg\max_{X_i \in \mathcal{D}} I(X_i, Y)$
$\mathcal{C}_{(1)} \leftarrow \{X^*\}$

// Step 2: Greedy selection of remaining $n-1$ features
for $t = 2$ **to** n **do**
    // Maximize the mRMR objective, g (MID or MIQ)
    $X^* \leftarrow \arg\max_{X_i \in \mathcal{D} \setminus \mathcal{C}_{(t-1)}} g(X_i; \mathcal{C}_{(t-1)}, Y)$
    $\mathcal{C}_{(t)} \leftarrow \mathcal{C}_{(t-1)} \cup \{X^*\}$
return $\mathcal{C}_{(n)}$

$$\mathcal{C}_{(t)} = \mathcal{C}_{(t-1)} \cup \underset{X_i \notin \mathcal{C}_{(t-1)}}{\arg\max}\, g(X_i; \mathcal{C}_{(t-1)}, Y).$$

mRMR feature importance function, g

MID: (basically) Relevance - Redundancy

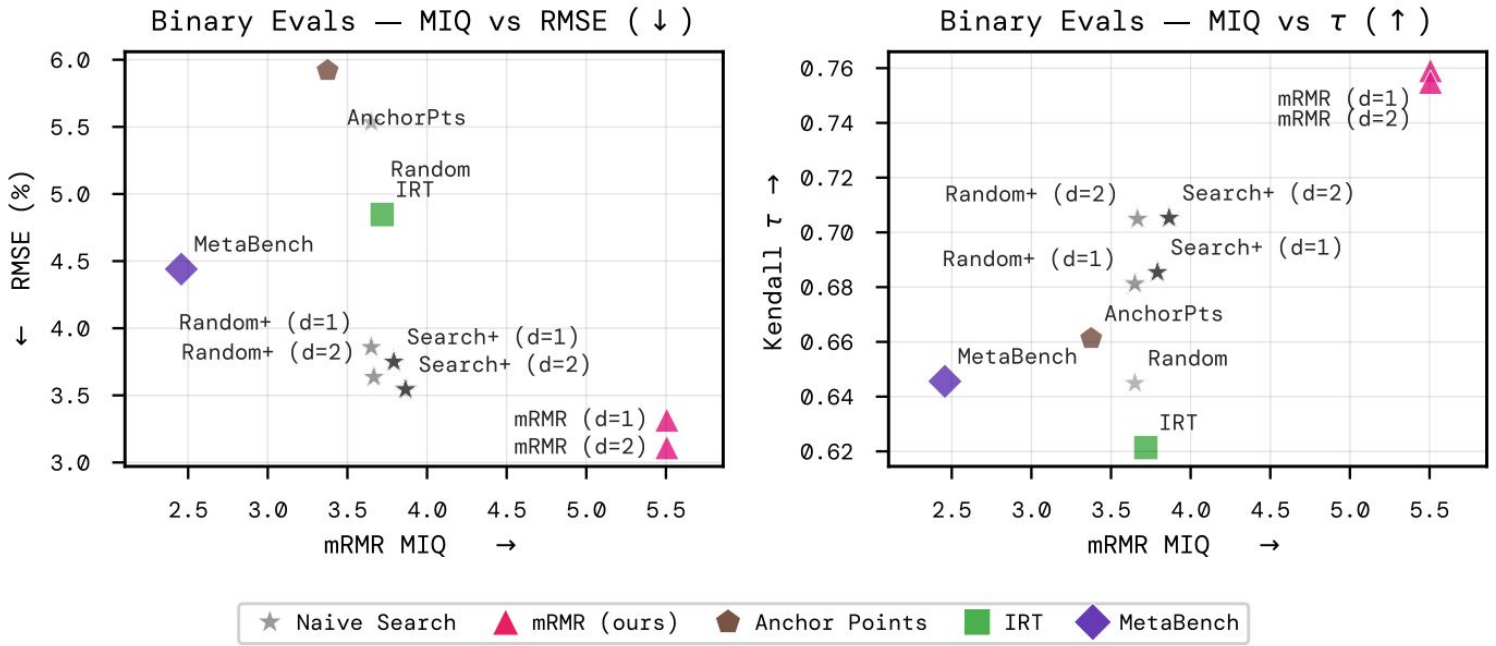
MIQ: (basically) Relevance / Redundancy <- this one works best in our experience

$$\text{(MI Difference)} \quad g^{\text{MID}}(X_i; \mathcal{C}_{(t-1)}, Y) = I(X_i, Y) - \frac{1}{|\mathcal{C}_{(t-1)}|} \sum_{X_j \in \mathcal{C}_{(t-1)}} I(X_j, X_i),$$

$$\text{(MI Quotient)} \quad g^{\text{MIQ}}(X_i; \mathcal{C}_{(t-1)}, Y) = I(X_i, Y) \Bigg/ \left(\frac{1}{|\mathcal{C}_{(t-1)}|} \sum_{X_j \in \mathcal{C}_{(t-1)}} I(X_j, X_i) \right)$$

The greedy approach *does* maximise the objective

(and this *does* help performance)



mRMR for coreset construction

Suppose we have N questions in a dataset and per-model per-question scores for M train/source models.

Represent each question (X_i, i=1,...,N) as $\mathbf{q}_i = [s(m_1, q_i), \ldots, s(m_M, q_i)] \in \mathbb{R}^M$

Represent the full-benchmark scores (Y) as $\bar{\mathbf{s}} = [\bar{s}(m_1, \mathcal{D}), \ldots, \bar{s}(m_M, \mathcal{D})] \in \mathbb{R}^M$.

Then we have M datapoints from the N+1 dimensional joint space

$$s(\cdot\,, q_1),\ s(\cdot\,, q_2),\ \ldots,\ s(\cdot\,, q_N),\ \bar{s}(\cdot\,, \mathcal{D}).$$

So we can estimate the MI between any two of these random variables

Coreset mRMR Binary MI calculation (Redundancy)

Redundancy terms compute I(X_i, X_j) = I(q_i, q_j) where q_i,q_j are binary vectors

$$I(Q_i, Q_j) = \sum_{q_i \in \{0,1\}} \sum_{q_j \in \{0,1\}} p(q_i, q_j) \log \left(\frac{p(q_i, q_j)}{p(q_i) p(q_j)} \right)$$

Use empirical proportions of 1s and 0s to estimate p(q_i), p(q_j), p(q_i,q_j)

$$p(\mathbf{q}_i = a, \mathbf{q}_j = b) = \frac{1}{M} \sum_{m=1}^{M} \mathbb{I}(q_i^{(m)} = a) \mathbb{I}(q_j^{(m)} = b),$$

$$p(\mathbf{q}_i = a) = \frac{1}{M} \sum_{m=1}^{M} \mathbb{I}(q_i^{(m)} = a), \quad \text{and} \quad p(\mathbf{q}_j = b) = \frac{1}{M} \sum_{m=1}^{M} \mathbb{I}(q_j^{(m)} = b).$$

Coreset mRMR MI calculation (Relevance)

In general, Y= $\bar{\mathbf{s}} = [\bar{s}(m_1, \mathcal{D}), \ldots, \bar{s}(m_M, \mathcal{D})] \in \mathbb{R}^M$ is not a binary vector, so for relevance MI calculations I(X_i,Y) = I(q_i, \bar{s}), we use a discrete-continuous MI estimator (details in appendix)

Also, if our benchmark questions don't just report binary scores, we'll have to use a continuous-continuous MI estimator (KSG, or PCA-corrected KSG) (details in appendix)

NOTE: these MI estimators work using k-NN, so we've introduced a hyperparameter that needs to be tuned: k=3,4,**5**,6,7,8,9

Predictions using ridge regression

Regular ridge (lambda chosen via cross validation)

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w},w_0} \left\{ ||\mathbf{y} - (w_0 + \mathbf{X}\mathbf{w})||_2^2 + \lambda ||\mathbf{w}||_2^2 \right\} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y},$$

Polynomial kernel ridge

$$\hat{\boldsymbol{\alpha}} = \arg\min_{\boldsymbol{\alpha}} \{ ||\mathbf{y} - K\boldsymbol{\alpha}||_2^2 + \lambda \boldsymbol{\alpha}^T K \boldsymbol{\alpha} \} = (K + \lambda\mathbf{I})^{-1}\mathbf{y}.$$

$$[K]_{ij} = \kappa(X_i, X_j) = (\langle X_i, X_j \rangle + 1)^d$$

b/c of kernel trick, this allows us to regress using the product of scores on all *groups of questions up to size d*

(d=1 is the same as regular ridge but with rescaled w_0)

Other methods

- **Random**: sample C (size n) at random, predict mean score
- **Random+**: sample C at random, predict w (kernel) ridge
- **Search**+: sample 10,000 Cs at random, fit (kernel) ridge on 80% of training data, choose C that leads to lowest MAE and refit
- **Lasso**: increase regularisation (from 0) until you only have n nonzero coefficients
- **Anchor points**: perform K-Means on $\mathbf{q}_i = [s(m_1, q_i), \ldots, s(m_M, q_i)] \in \mathbb{R}^M$
  - Predicted weighted mean of K=n centroids with weights given by cluster sizes
- **IRT – TinyBenchmarks:** $p_{i,m} = \mathbb{P}(s(m, q_i) = 1 \mid \theta_m, \alpha_i, \beta_i) = 1/\left(1 + \exp(-\alpha_i^T \theta_m + \beta_i\right)$
  - Select C by running K-Means on $(\hat{\alpha}_i, \hat{\beta}_i) \in \mathbb{R}^{p+1}$ $\theta_m, \alpha_i \in \mathbb{R}^p$ and $\beta_i \in \mathbb{R}$
  - p-IRT: estimate \theta* for a new test model m* (via max lik.) (CV over p=1,5,10)
    - Then use $p_{i,m}$ as a stand-in for the non-coreset questions
  - gp-IRT: convex combination of p-IRT and anchor-points style prediction (with cluster-sizes w_i)

$$f_{\mathcal{C}}^{\text{gp-IRT}}([s(m^*, q_i)]_{q_i \in \mathcal{C}}) = \lambda \sum_{i:\ q_i \in \mathcal{C}} w_i s(m^*, q_i) + (1 - \lambda) f_{\mathcal{C}}^{\text{p-IRT}}([s(m^*, q_i)]_{q_i \in \mathcal{C}}).$$

Experiment Datasets

Binary: 13 datasets with between 83 and 441 models

Continuous:

- 2 datasets, one with BERTScore + Rouge-L, one with those + FKGL,
- 21 models x 4 temperatures

Pass@k:

- 4 python benchmarks, 128 generations per question, construct coresets with pass@1 data, predict on all other pass@k for k=1,2,4,8,16,32,64, 24
- 25 models (9 x 3 temps minus a couple failed runs)

M = 15, 30, 50 (or 16, 32, 52 for continuous) using stratified sampling (with 5 random seeded train-test splits)

coreset sizes = 5%, 10%, 15%

Metrics

- **MAE, RMSE** (normalised by stddev of each benchmarks' summary scores)
- **Spearman rho**: Pearson corr. on rank vectors R (ith entry = rank of model i)

$$\rho(\hat{R}_{\mathcal{D}}, R_{\mathcal{D}}) = \frac{\text{Cov}(\hat{R}_{\mathcal{D}}, R_{\mathcal{D}})}{\text{StdDev}(\hat{R}_{\mathcal{D}})\text{StdDev}(R_{\mathcal{D}})} \in [-1, 1].$$

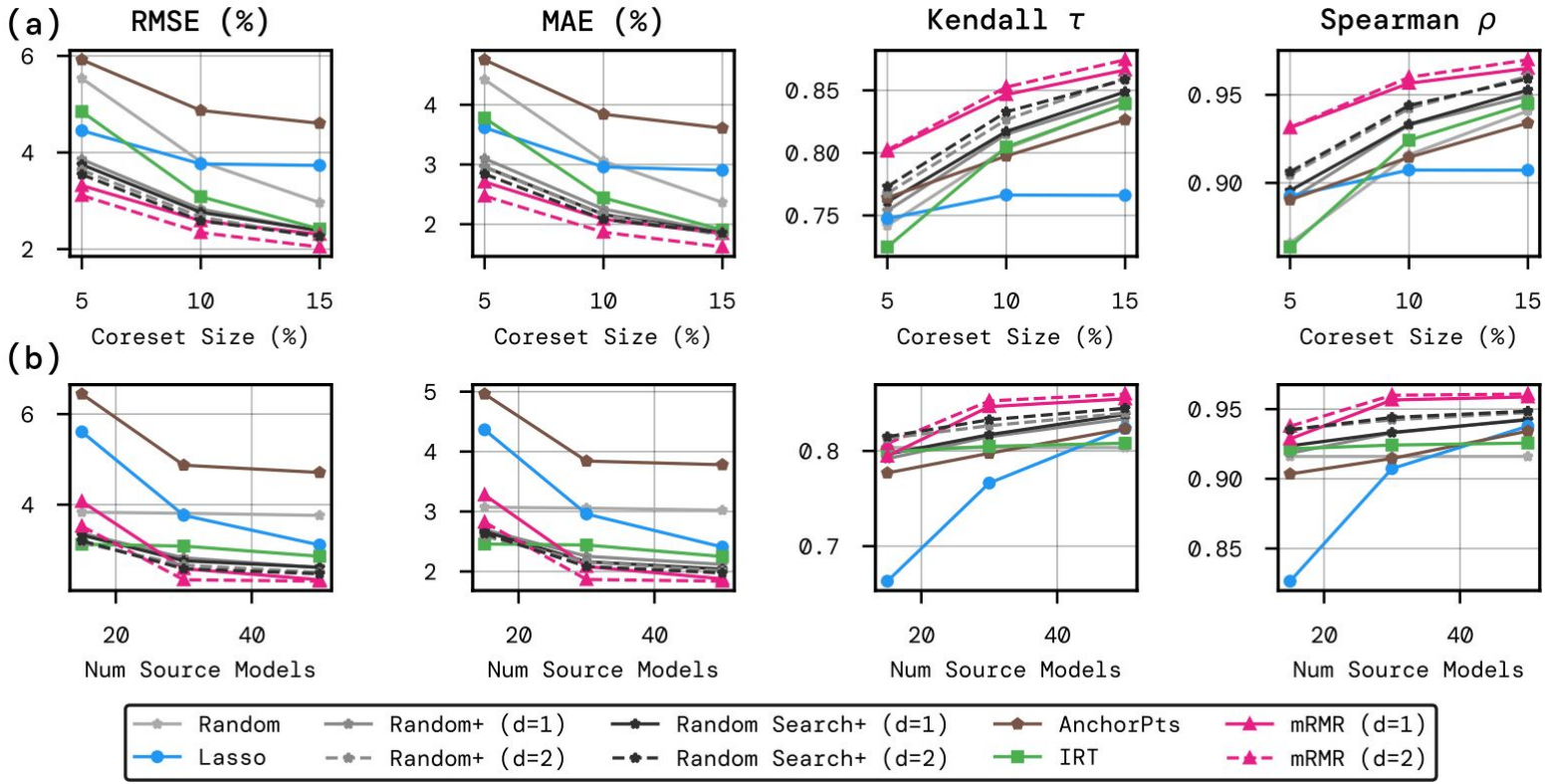
- **Kendall tau**: proportion of concordant rankings (same rel. order) vs discordant

$$\tau(\hat{R}_{\mathcal{D}}, R_{\mathcal{D}}) = \frac{1}{\binom{n}{2}} \sum_{1 \le m_1 < m_2 \le M'} \text{sign}(\hat{R}_{\mathcal{D}}^{m_1} - \hat{R}_{\mathcal{D}}^{m_2})\text{sign}(R_{\mathcal{D}}^{m_1} - R_{\mathcal{D}}^{m_2}) \in [-1, 1].$$

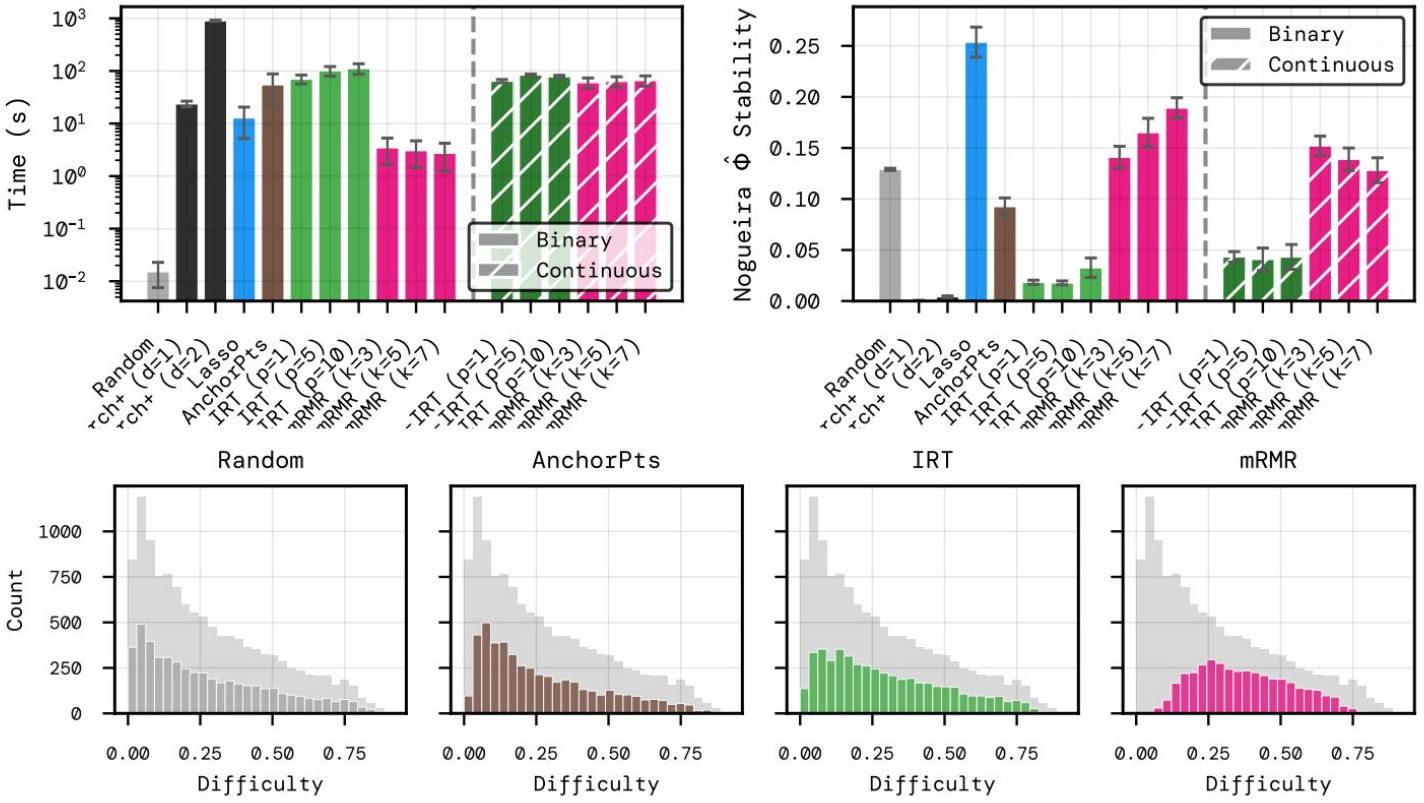
Note that a value of τ means the proportion of incorrectly ranked pairs is $(1 - \tau)/2$.

- **Phi-hat stability**: Complicated, but 0 means random coresets, 1 means always picks the same coreset
- (**Hamming stability**: avg. L1 distance between coresets as represented by binary vectors of length N (containing n 1s))

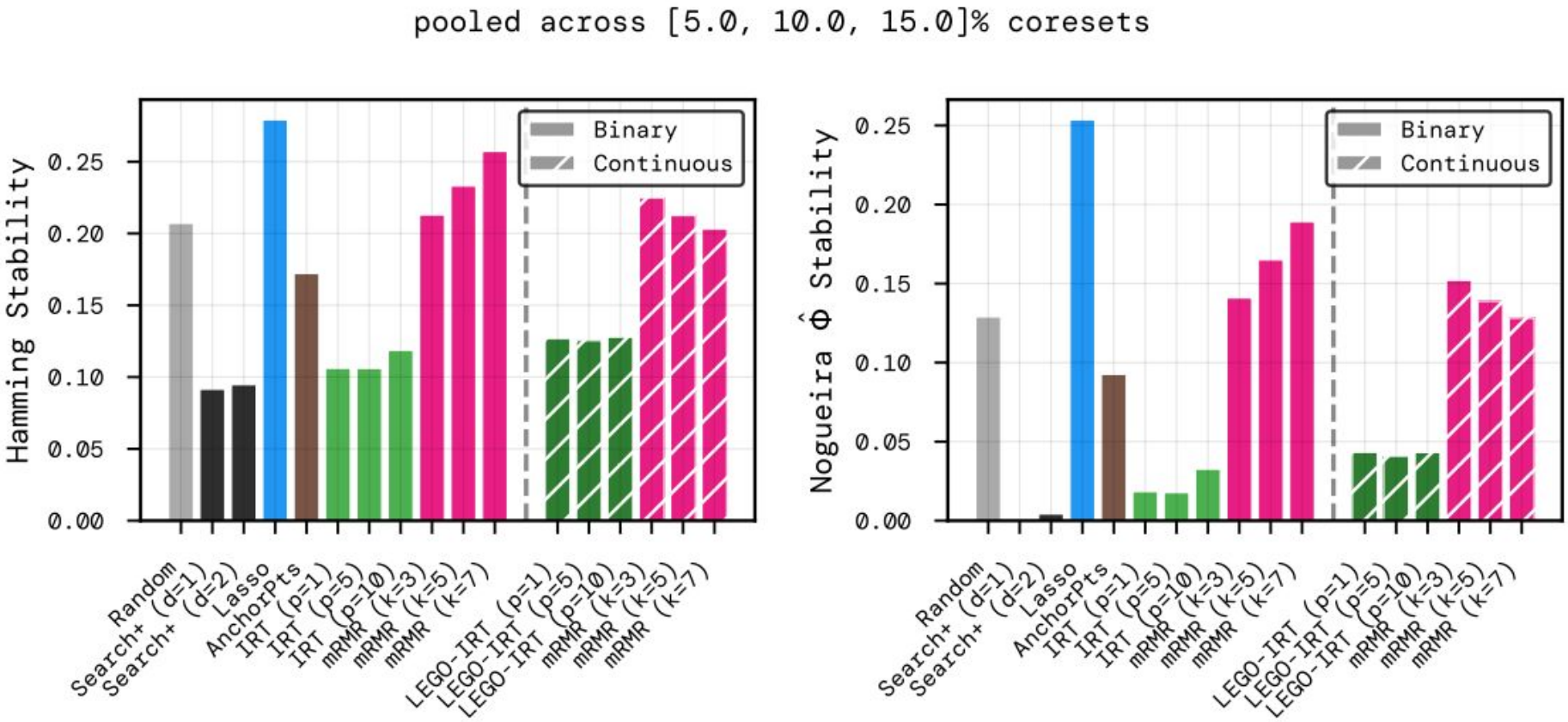
Binary results



Binary analysis

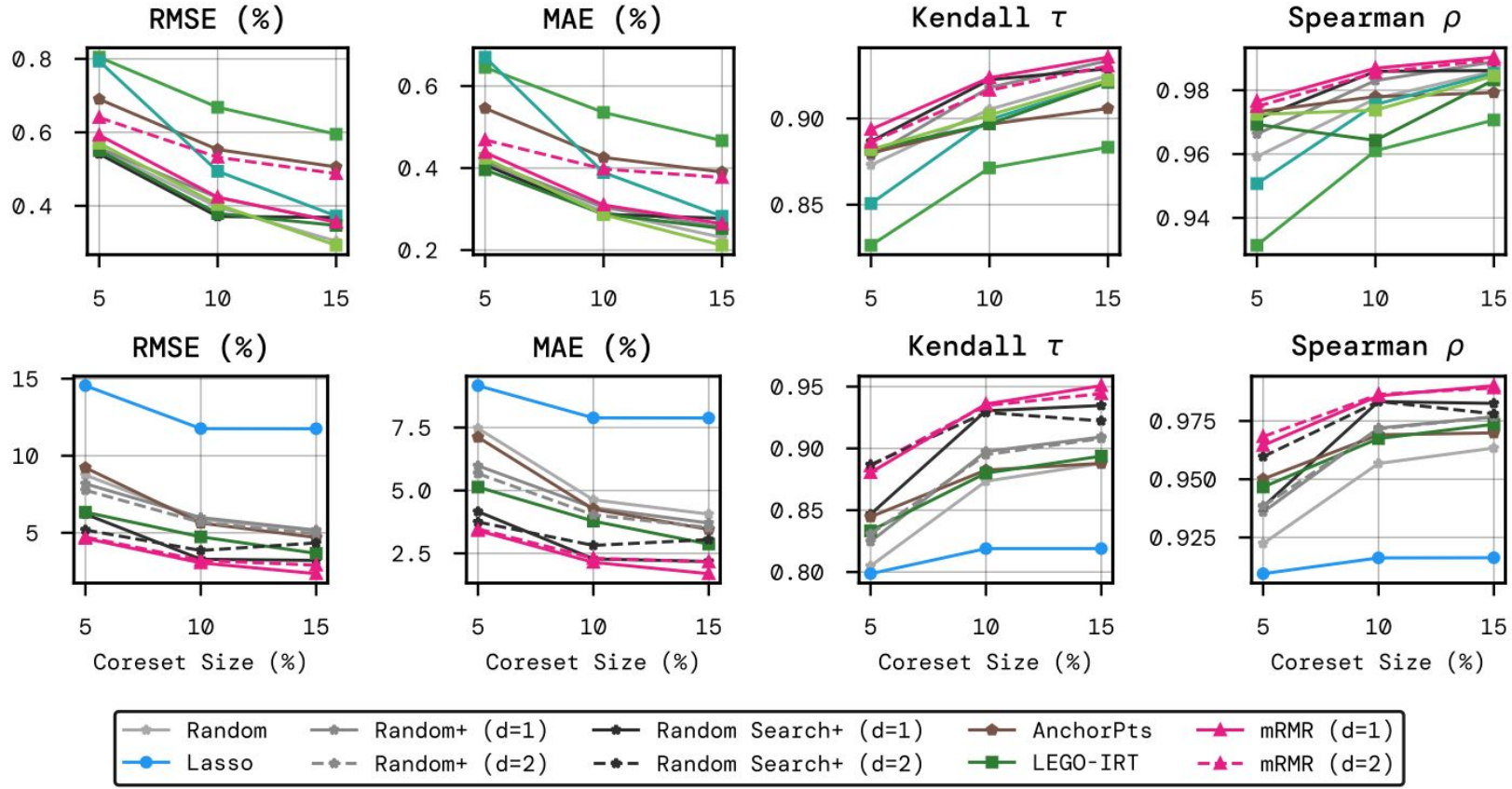


(w/ Hamming stability too)

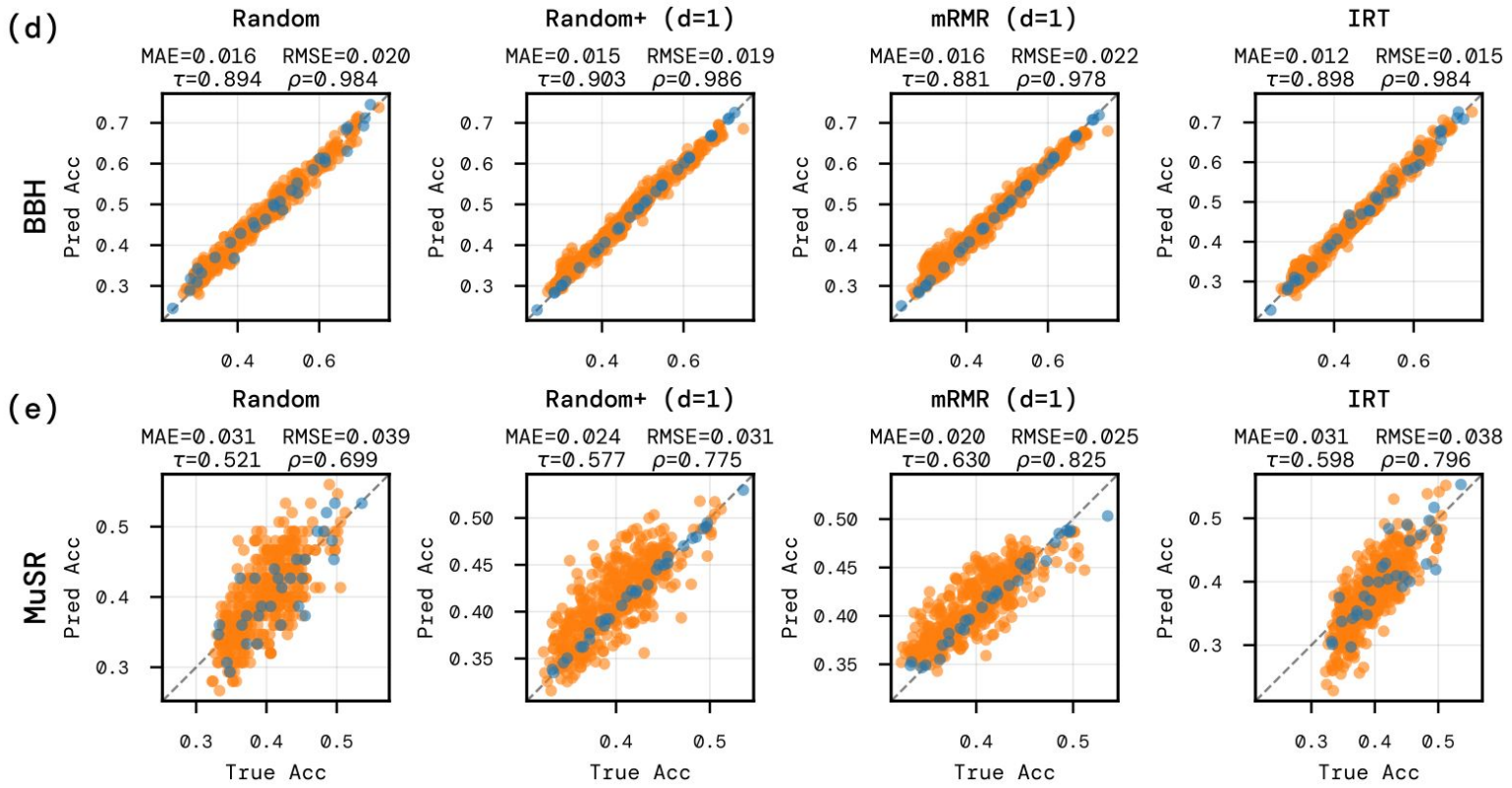


Continuous & Pass@K results

Fig



Limitations



[Conclusion here]

Note we don’t try OOD prediction or dynamic/active coreset prediction (future work?) but we don’t really care to anyway

We also just use per-model per-question evals (many other methods are a lot more complicated)

Main takeaways:

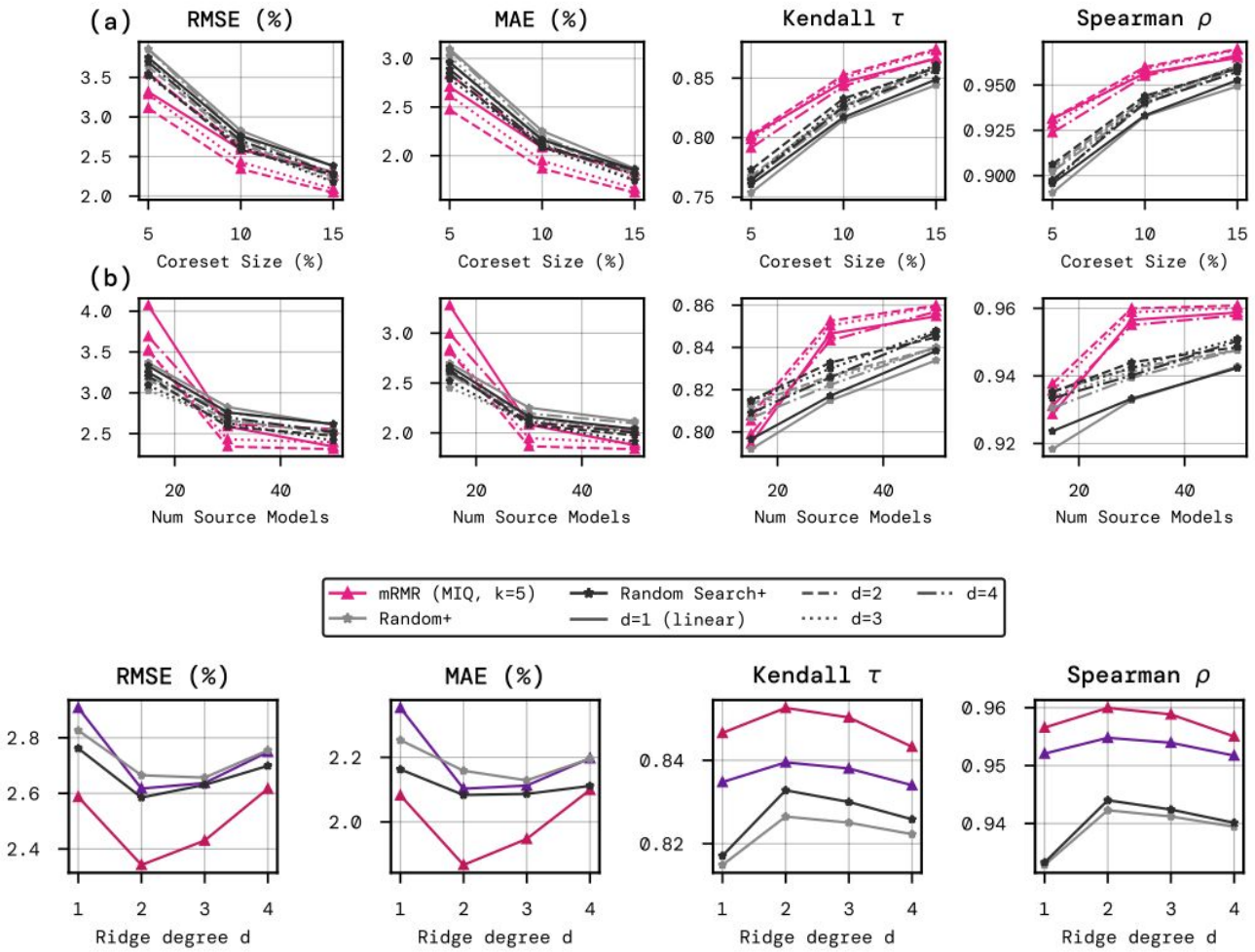
-feature-selection+simple regression framing

- we don’t need many source models

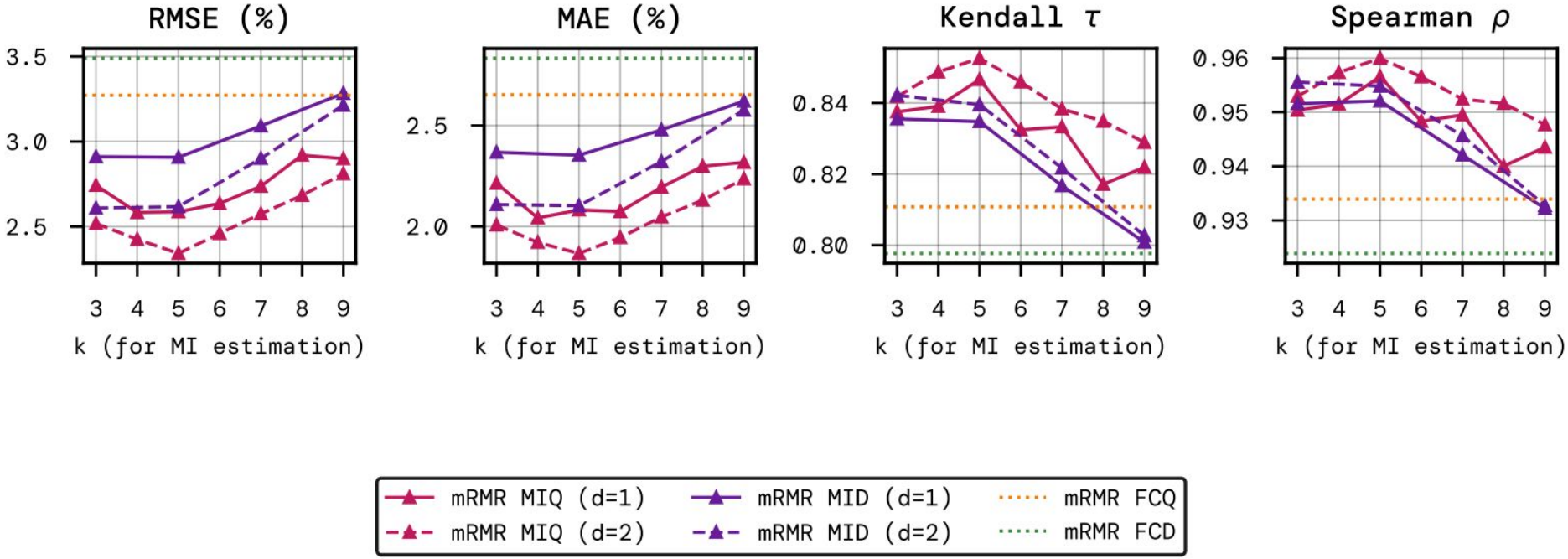
- lower variance predictions -> better ranking predictions

Ablations: kernel ridge degree, d

(binary datasets)



mRMR k (for MI estimation), and MIQ vs MID (vs FCQ/FCD)

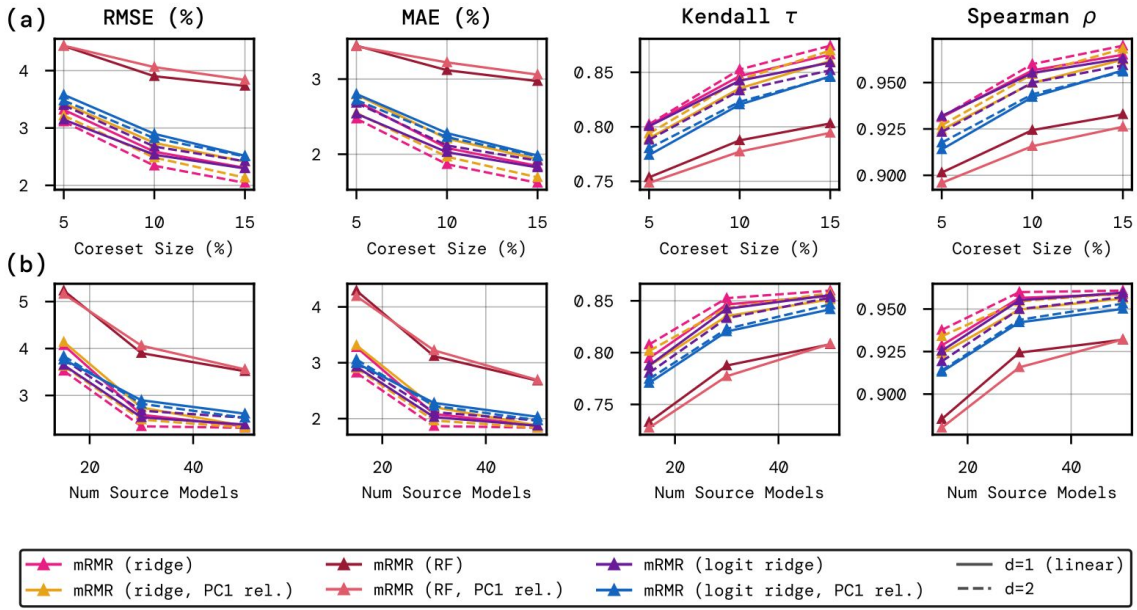


Prediction strategy and relevance target Y

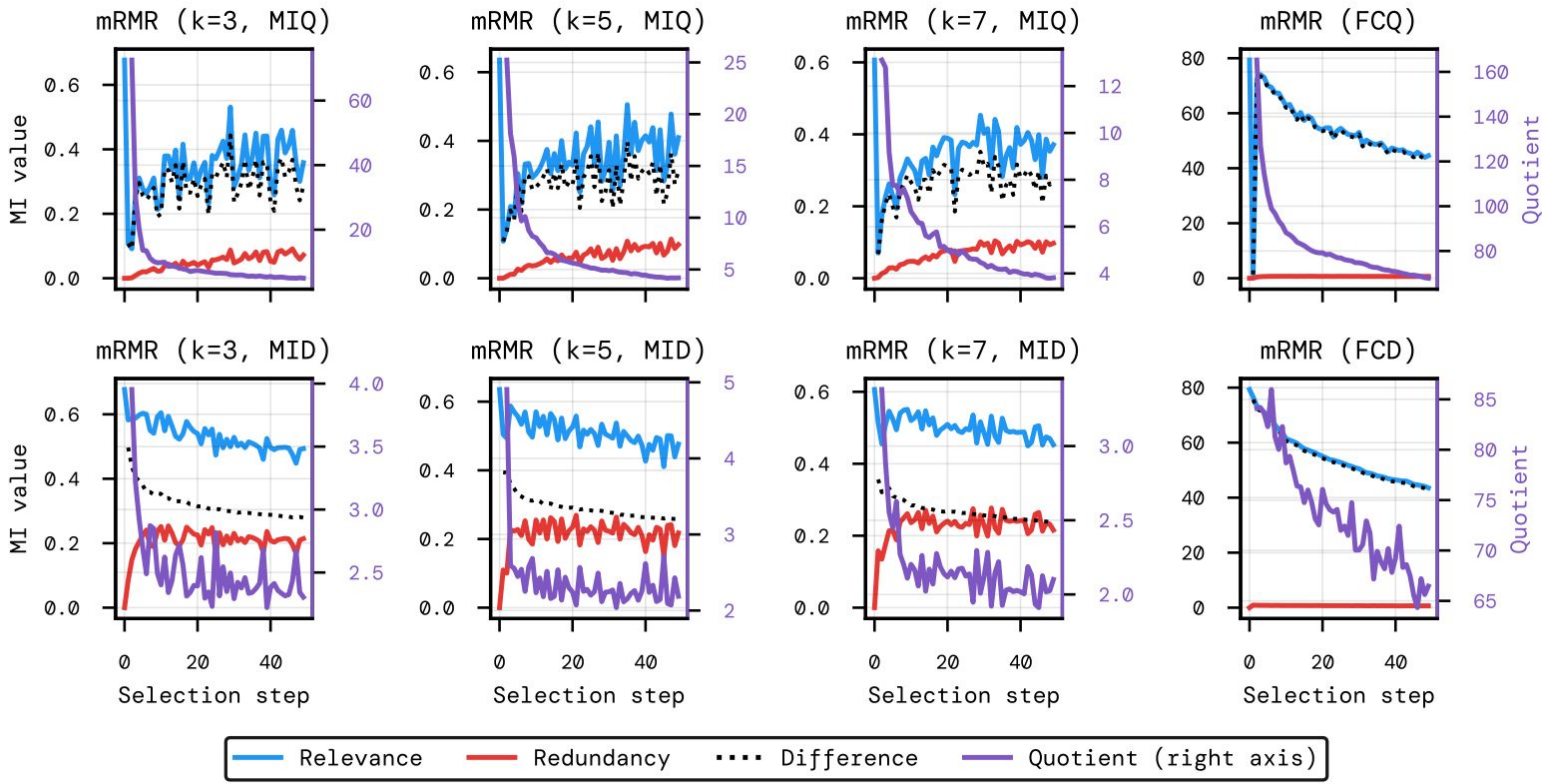
PC1 rel. Means instead of using Y= $\bar{\mathbf{s}} = [\bar{s}(m_1, \mathcal{D}), \ldots, \bar{s}(m_M, \mathcal{D})] \in \mathbb{R}^M$.

for mRMR relevance I(X_i,Y), we use the dot product of $\mathbf{q}_i = [s(m_1, q_i), \ldots, s(m_M, q_i)] \in \mathbb{R}^M$

And the first principal component of all q_1,...,q_n (~'g factor')



mRMR coreset construction



Relevance & Redundancy of all methods' coresets